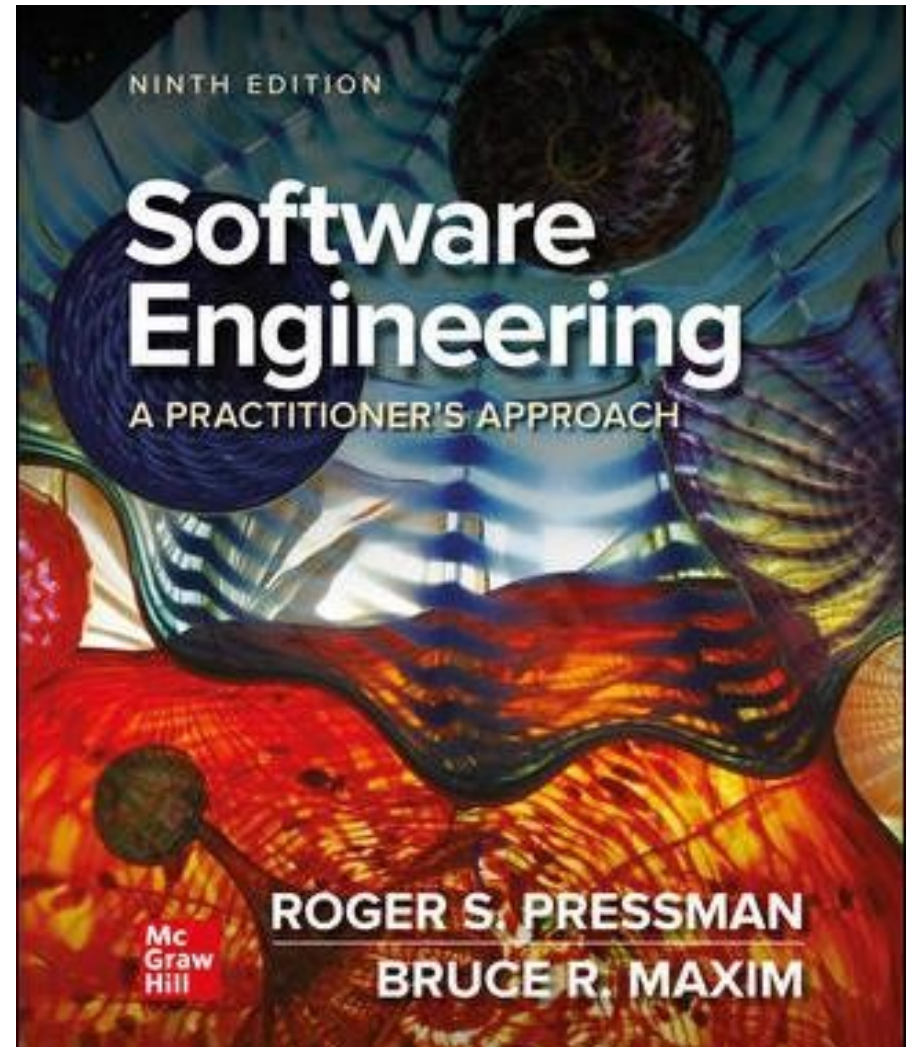


Chapter 1

Software and Software Engineering

Introduction





高楠

电子邮件: nannn.gao@gmail.com

职 称: 副教授

学 历: 博士

所学专业: 计算机科学与技术

研究方向: 普适计算、人机交互、情感计算、智能感知、智慧教育

个人主页: <https://nancygao.com/>

高楠，现任南开大学软件学院副教授，于2022年在澳大利亚皇家墨尔本理工大学（RMIT）计算机系获得博士学位，随后加入清华大学计算机系人机交互团队，担任水木学者博士后及助理研究员，并在澳大利亚新南威尔士大学（UNSW）计算机系做访问学者。近年来，在普适计算与人机交互领域发表论文16篇（一作论文8篇），并开源贡献了迄今规模最大、数据类型最丰富的学生心理感知数据集En-Gage。其研究成果曾被澳大利亚新闻广播公司（ABC News）专访及全球百余家媒体报道。曾获IMWUT (CCF-A) 杰出论文奖（一作）、Ubicomp '24 Workshop最佳论文奖（一作）、Ubicomp'20 最佳学术报告民选提名奖、国家优秀自费留学生奖、德国学术交流中心（DAAD）AINet Fellow等荣誉。此外，现担任普适计算与人机交互领域顶级期刊IMWUT副主编、中国计算机协会人机交互委员会执行委员、中国图像图形学会人机交互委员会委员等职务。主持中国博士后科学基金面上资助、国自然青年基金项目等。

本人研究旨在利用技术手段量化个体行为与心理特质，并探索合适的交互干预策略，以促进人类福祉与提升幸福感。对计算机科学、心理学与教育学的交叉领域方面尤为关注，特别是大模型在心理健康、情感陪伴和家庭教育领域的应用研究。欢迎对相关方向感兴趣的学生通过电子邮件联系我。更多信息请访问个人主页（<https://nancygao.com/>）。

Grading Policy

Type	Component	Weight	Details
Individual	Final Examination	50%	A comprehensive exam assessing understanding of core software engineering principles, methodologies, and theoretical knowledge covered throughout the course.
Group	Group Project (动感地带AI高校创智计划)	35%	A hands-on project evaluating the entire software lifecycle: requirement elicitation, system design, implementation, testing, and deployment. Key focus on teamwork, tool usage, and creating a functional, valuable product.
	Conference-Style Paper	10%	A complete academic paper formatted for a target conference. Must include abstract, introduction, methodology, results, discussion, and conclusions. Evaluates ability to communicate research and technical work formally.
Individual	Class Participation & Quizzes	5%	Based on active contribution to lectures, in-class exercises. Attendance is a prerequisite for earning these points.

Why did you choose Software Engineering?

- Job Security?
 - High Salary?
 - To Build Things?
 - Passion for Technology?
-

The New Reality

- What happens when we have ChatGPT?
- ...when GitHub Copilot writes code for us?
- ...when AI can generate an entire app in seconds?

Does our "golden ticket" still exist?

A Question We All Share

- Let's hear from an industry expert and educator.
- Before we watch, think about:

What is the core value of a software engineer in the AI era?
What must we learn to stay essential?

【TED演讲】人工智能时代的软件工程学习（12分钟）

Key Takeaways from the Video

■ AI's Limitations:

Hallucinates | No Context | Can't Prioritize | Unreliable

■ Our Uniqueness:

Strategic Thinking | Vision & Definition | Responsibility | Empathy |
Collaboration

■ Our New Role:

From Coder → to → Orchestrator & Leader

The Goal of This Course

- **Define Vision** (Requirements Engineering)
 - **Architect Systems** (Software Design)
 - **Make Trade-offs** (Project Management)
 - **Collaborate Effectively** (Teamwork)
 - **Build Responsibly** (Ethics)
 - **Leverage AI** (Modern Tooling)
-

Nature of Software – Defining Software

Software is:

- 1) Instructions (computer programs) that when executed provide desired features, function, and performance;*
- 2) Data structures that enable the programs to adequately manipulate information.*
- 3) Documentation that describes the operation and use of the programs.*

What is Software?

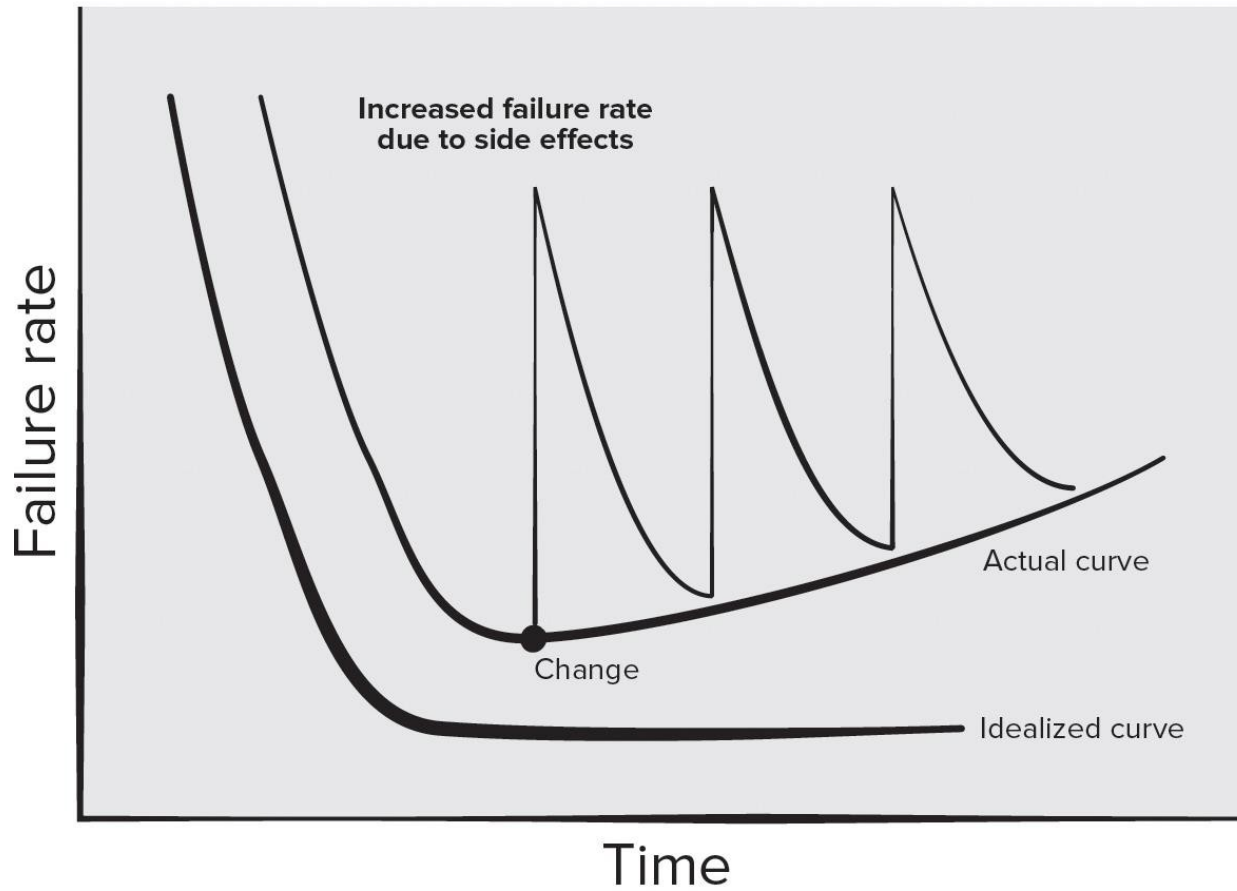
- *Software is developed or engineered it is not manufactured in the classical sense.*
- *Software doesn't "wear out" but is does deteriorate.*
- *Although the industry is moving toward component-based construction, most software continues to be custom-built.*

Software Application Domains

- System software.
- Application software.
- Engineering/Scientific software.
- Embedded software.
- Product-line software.
- Web/Mobile applications.
- AI software (robotics, neural nets, game playing).

Wear versus Deterioration

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.



[Access the text alternative for slide images.](#)

Legacy Software

Why must software change?

- Software must be *adapted* to meet the needs of new computing environments or technology.
- Software must be *enhanced* to implement new business requirements.
- Software must be *extended* to make it interoperable with other more modern systems or databases.
- Software must be *re-architected* to make it viable within a network environment.

Defining the Discipline

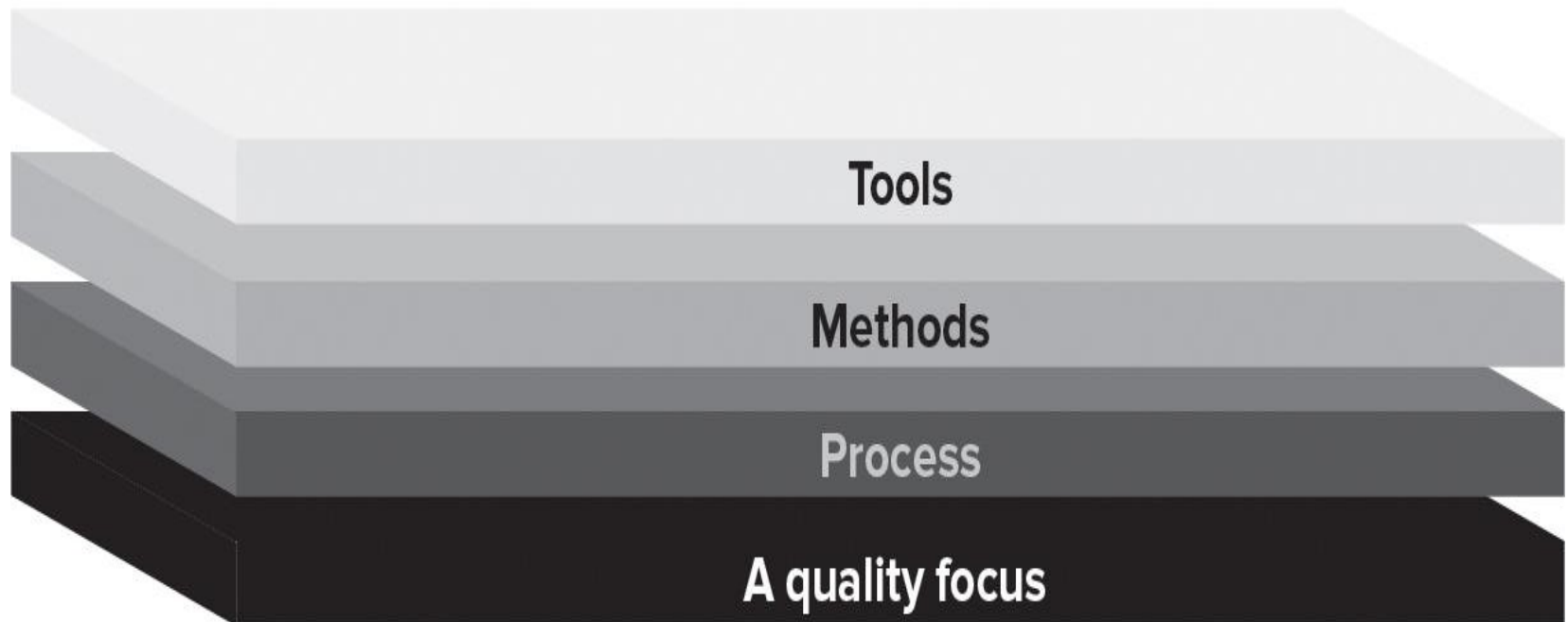
The IEEE definition:

Software Engineering:

1. *The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software.*
2. *The study of approaches as in (1).*

Software Engineering Layers

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.



Process Framework Activities

Communication.

Planning.

Modeling.

- Analysis of requirements.
- Design.

Construction:

- Code generation.
- Testing.

Deployment.

Umbrella Activities

- Software project tracking and control.
- Risk management.
- Software quality assurance.
- Technical reviews.
- Measurement.
- Software configuration management.
- Reusability management.
- Work product preparation and production.

Process Difference Requiring Adaptation

- Overall flow of activities, actions, and tasks and the interdependencies among them.
- Degree to which actions and tasks are defined within each framework activity.
- Degree to which work products are identified and required.
- Manner which quality assurance activities are applied.
- Manner in which project tracking and control activities are applied.
- Overall degree of detail and rigor with which the process is described.
- Degree to which the customer and other stakeholders are involved with the project.
- Level of autonomy given to the software team.
- Degree to which team organization and roles are prescribed.

Essence of Software Engineering Practice

In the classic book *How to Solve It* (1945) George Polya outlined the essence of problem solving:

1. *Understand the problem* (communication and analysis).
2. *Plan a solution* (modeling and software design).
3. *Carry out the plan* (code generation).
4. *Examine result for accuracy* (testing & quality assurance).

Understand the Problem

- *Who has a stake in the solution to the problem?*

That is, who are the stakeholders?

- *What are the unknowns?*

What data, functions, and features are required to properly solve the problem?

- *Can the problem be compartmentalized?*

Is it possible to represent smaller problems that may be easier to understand?

- *Can the problem be represented graphically?*

Can an analysis model be created?

Plan a Solution

- *Have you seen similar problems before?*

Are there patterns that are recognizable in a potential solution? Is there existing software that implements the data, functions, and features that are required?

- *Has a similar problem been solved?*

If so, are elements of the solution reusable?

- *Can subproblems be defined?*

If so, are solutions readily apparent for the subproblems?

- *Can you represent a solution in a manner that leads to effective implementation?*

Can a design model be created?

Carryout the Plan

- *Does **the solution conform to the plan**?*

Is source code **traceable** to the design model?

- *Is each component part of the solution provably correct?*

Has the design and code been reviewed, or better, have **correctness proofs** been applied to algorithm?

Examine the Result

- *Is it possible to test each component part of the solution?*

Has a reasonable testing strategy been implemented?

- *Does the solution produce results, that conform to the data, functions, and features that are required?*

Has the software been **validated** against all stakeholder requirements?

Hooker's General Principles

for software engineering practice

1. *The Reason It All Exists* – provide value to users.
2. *KISS (Keep It Simple, Stupid!)* – design simple as it can be.
3. *Maintain the Vision* – clear vision is essential.
4. *What You Produce, Others Will Consume.*
5. *Be Open to the Future* - do not design yourself into a corner.
6. *Plan Ahead for Reuse* – reduces cost and increases value.
7. *Think!* – placing thought before action produce results.

How it all Starts – SafeHome Begins

Every software project is precipitated by some business need—

- The need to correct a defect in an existing application;
- The need to the need to adapt a ‘legacy system’ to a changing business environment;
- The need to extend the functions and features of an existing application, or
- The need to create a new product, service, or system.